课程介绍

1. matplotlib
2. numpy-快速计算库
3. pandas--数据处理的包

3.3 常见图形绘制

1 常见图形种类及意义

1. 折线图 `api：plt.plot(x, y)`
   - 特点: **能够显示数据的变化趋势，反映事物的变化情况。**
   - 关键词:变化

2. 散点图 `api：plt.scatter(x, y)`
   - 特点: **判断变量之间是否存在数量关联趋势,展示离群点(分布规律)**
   - 关键词:分布规律

3. 柱状图(条形图) `api：plt.bar(x, width, align='center', **kwargs)`
   - 特点: **绘制连离散的数据,能够一眼看出各个数据的大小,比较数据之间的差别。(统计/对比)**
   - 关键词:对比(比大小)

```
Parameters:
x : 需要传递的数据

width : 柱状图的宽度

align : 每个柱状图的位置对齐方式
    {‘center’, ‘edge’}, optional, default: ‘center’

**kwargs :
color:选择柱状图的颜色
```

4. 直方图 `api：matplotlib.pyplot.hist(x, bins=None)`
   - **特点：绘制连续性的数据展示一组或者多组数据的分布状况(统计)**
   - 关键词: 统计

```
Parameters:
x : 需要传递的数据
bins : 组距
```

5. 饼图 `api：plt.pie(x, labels=,autopct=,colors)`
   - 特点: **描述数据占比**
   - 关键词: 占比

```
Parameters:
x:数量，自动算百分比
labels:每部分名称
autopct:占比显示指定%1.2f%%
colors:每部分颜色
```

Numpy

- 高速计算库

Numpy介绍（）

- num:Numerical---数值化
- py: python

特点:

- **用于快速处理任意维度的数组**
- Numpy **支持常见的数组和矩阵操作**
- Numpy **使用ndarray对象来处理多维数组**

矩阵:二维数组

- **矩阵一定是二维数组，二维数组不一定是矩阵**

2 ndarray介绍

- n--多个
- d-dimensional--维度
- array--数组

N维数组类型ndarray，描述相同类型的'Items'集合

3 ndarray与Python原生list运算效率对比

机器学习的最大特点就是大量的数据运算

4 ndarray的优势

1. 内存块风格
   - 由于ndarray数据类型相同,可以直接读取
   - list由于数据类型不同,通过地址查找
2. 并行计算(类比GPU)
3. 效率远高于原生python
   - Numpy底层使用C语言编写

4.2 N维数组-ndarray

ndarray的属性

属性名字	属性解释
ndarray.shape	形状

属性名字	属性解释
ndarray.ndim	数组的维数(2,3)
ndarray.size	数组元素个数
ndarray.dtype	**类型**
ndarray.itemsize	一个元素的长度

2 ndarray的形状

3 ndarray的类型

创建数组的时候指定类型

- 若不指定，整数默认int64，小数默认float64

4.3 基本操作

生成数组的方法

1. **生成0-1 数组**
   - `np.ones(shape, dtype)`
   - `np.zeros(shape, dtype)`

   应用场景:生成一个表格
2. **从现有数组生成**
   - `np.array()` --深拷贝
   - `np.asarray()` --浅拷贝 (生成快捷方式)
3. **生成固定范围数组**

   `np.linspace (start, stop, num, endpoint)`
   - 创建等差数组 – 指定数量
   - 参数:
     1. start:序列的起始值
     2. stop:序列的终止值
     3. num:要生成的等间隔样例数量，默认为50
     4. endpoint:序列中是否包含stop值，默认为ture

   `np.arange(start,stop, step, dtype)`
   - 创建等差数组 – 指定步长
   - 参数:
     - step:步长,默认值为1

   `np.logspace(start,stop, num)`
   - 创建等比数列
   - 参数:
     - num:要生成的等比数列数量，默认为50
     - base 以几为底
4. **生成随机数组**
   - np.random模块

正太分布

什么是方差标准差

- 方差:描述数据的离散程度

$$\sigma^2 = \frac{(x_1 - M)^2 + (x_2 - M)^2 + (x_3 - M)^2 + \ldots\ldots (x_n - M)^2}{n}$$

方差/标准差大-->离散程度大

方差/标准差小-->离散程度小

N(μ，σ)

- u: 均值 决定数据的位置
- σ :标准差 决定数据的离散程度
  - σ 大 数据样式矮胖
  - σ 小 数据是瘦高

标准正态分布 N(0,1)

- `np.random.randn(*d0, d1, …, dn*)`
  - 功能：从标准正态分布中返回一个或多个样本值
- **`np.random.normal(loc=0.0, scale=1.0, size=None)`**
  1. loc：float
     - 此概率分布的均值（对应着整个分布的中心centre）
  2. scale：float
     - 此概率分布的标准差（对应于分布的宽度，scale越大越矮胖，scale越小，越瘦高）
  3. size：int or tuple of ints
  - 输出的shape，默认为None，只输出一个值
- `np.random.standard_normal(*size=None*)`
  - 返回指定形状的标准正态分布的数组。

均匀分布

- `np.random.rand(d0, d1, ..., dn)`
  - 返回[0.0，1.0)内的一组均匀分布的数。
- **`np.random.uniform(low=0.0, high=1.0, size=None)`**
  - 功能：从一个均匀分布[low,high)中随机采样，注意定义域是左闭右开，即包含low，不包含high.
  - 参数介绍:
    - low: 采样下界，float类型，默认值为0；
    - high: 采样上界，float类型，默认值为1；
    - size: 输出样本数目，为int或元组(tuple)类型，例如，size=(m,n,k), 则输出mnk个样本，缺省时输出1个值。
  - 返回值：ndarray类型，其形状和参数size中描述一致。
- `np.random.randint(low, high=None, size=None, dtype='l')`
  - 从一个均匀分布中随机采样，生成一个整数或N维整数数组，
  - 取数范围：若high不为None时，取[low,high)之间随机整数，否则取值[0,low)之间随机整数。

2 数组的索引、切片

一维、二维、三维的数组如何索引?

- 直接进行索引,切片
- 对象[:, :] -- 先行后列

3 形状修改

size必须相同

- `ndarray.reshape(shape, order)`
  - 复制原数值,不修改原值
- `ndarray.resize(new_shape)`
  - 修改原值
  - 自动计算几行

ndarray.T

- 数组的转置
- 将数组的行、列进行互换

4 类型修改

- ndarray.astype(type)
- 返回修改了类型之后的数组
- ndarray.tostring([order])或者ndarray.tobytes([order])

5 数组的去重

- np.unique()

ndarray运算

1 逻辑运算

布尔索引

2 通用判断函数

`np.all()`

- all里面所有数据都为True 返回True
- all里面有一个为False返回False

`np.any()`

- any()里面只要一个True 返回True
- any里面全是False返回False

3 np.where（三元运算符）

np.where(逻辑条件,符合条件执行,不符合条件执行)

4 统计运算

4.1 统计指标

- min()--最小值
- max()--最大值

- median()--中间值
- mean()--平均值
- std()--标准差
- var()--方差
- np.argmax()--最大值下标
- np.argmin()--最小值下标

4.5 数组间运算

1 数组与数的运算

arr+1

2 数组与数组的运算

- 要求数据形状相同可以执行计算
- 如果形状不同符合广播机制
  - 数组某一维度等长
  - 其中一个维度为1

4.6 数学：矩阵

1 矩阵和向量

矩阵：是一个`二维数组`

矩阵元素：`item`

1.2 向量

列向量

3 矩阵向量乘法

矩阵乘法遵循准则：

(M行, N列)*(N行, L列) = (M行, L列)

5 矩阵乘法的性质

- 矩阵的乘法不满足交换律：A×B≠B×A
- 矩阵的乘法满足结合律。即：A×（B×C）=（A×B）×C
- 单位矩阵:方阵 I / E 可看成1

6 逆、转置

A*B=I :A矩阵和B互逆矩阵

7.1 矩阵乘法api

- `np.matmul`
- np.matmul中禁止矩阵与标量的乘法
- np.dot

Pandas

- pan :panal 面板

- d:data 数据
- as: analysis 分析

以Numpy为基础，借力Numpy模块在计算方面性能高的优势

- **基于matplotlib，能够简便的画图**
- **独特的数据结构**

2 为什么使用Pandas

1. 图表可读性强
2. 数据处理方便
3. 读取文件方便
4. 封装了Matplotlib、Numpy的画图和计算

5.2 Pandas数据结构

1. Series
2. Dateframe
3. panel(MultiIndex)

1.Series

主要由一组数据和与之相关的索引两部分构成。

1.1 Series的创建

`pd.Series(data,index=,dtype=)`

- 参数：
  - data：传入的数据，可以是ndarray、list等
  - index：索引，必须是唯一的，且与数据的长度相等。如果没有传入索引参数，则默认会自动创建一个从0-N的整数索引。
  - dtype：数据的类型

1.2 Series的属性

属性名字	属性解释
对象.index	索引
对象.values	值内容

2.DataFrame

- 行索引
- 列索引
- axis 轴

2.1 DataFrame的创建

`pd.DataFrame(data=None, index=None, columns=None)`

- 参数：
  - index：行索引
  - columns： 列索引

2.2 DataFrame的属性

属性名字	属性解释
对象.shape	几行几列
对象.index	行索引列表
对象.columns	列索引列表
对象.values	array的值
对象.T	转置
对象.head()	前5行内容
对象.tail()	后5行内容

2.3 DatatFrame索引的设置

修改行列索引值

- `对象.index=`
  - 不能修改单个

重设索引

- `对象.reset_index(drop=)`
  - drop:默认为False，不删除原来索引，如果为True,删除原来的索引值

2.3.3 以某列值设置为新的索引

- `set_index(key,drop=)`
  - 设置多个索引，以年和月份

3.MultiIndex与Panel

3.1.1 multiIndex的特性

- MultiIndex是三维的数据结构

对象.index.names

对象.index.levels

3.1.2 multiIndex的创建

```
arrays = [[1, 1, 2, 2], ['red', 'blue', 'red', 'blue']]
pd.MultiIndex.from_arrays(arrays, names=('number', 'color'))
```

3.2 Panel

class `pandas.Panel`(*data=None*, *items=None*, *major_axis=None*, *minor_axis=None*)

- 作用：存储3维数组的Panel结构
- 参数：
  - **data** ：ndarray或者dataframe
  - **items** ：索引或类似数组的对象，axis=0
  - **major_axis** ：索引或类似数组的对象，axis=1
  - **minor_axis** ：索引或类似数组的对象，axis=2

5.3 基本数据操作

1 索引操作

1.1 直接使用行列索引(先列后行)

1.2 结合loc或者iloc使用索引

- 对象.loc[行名,列名]
- 对象.iloc[行下标,列下标]

1.3 使用ix组合索引

组合名称和下标值索引内容

2 赋值操作

- data['列名']=

3 排序

3.1 DataFrame排序

1. 根据值排序

使用`对象.sort_values(by=, ascending=)`

- 单个键或者多个键进行排序,
- 参数:
  - by：指定排序参考的键
  - ascending:默认升序
    - ascending=False:降序
    - ascending=True:升序
2. 根据索引排序

   `对象.sort_index(ascending=)`

3.2 Series排序

`对象.sort_values(ascending=True)`

`对象.sort_index(ascending= )`

5.4 DataFrame运算

1 算术运算

- `对象.add()===对象+`
- `对象.sub()====对象-`

2 逻辑运算

2.1 逻辑运算符号

对象[逻辑判断]

逻辑判断是一个bool索引

2.2 逻辑运算函数

1. 范围查

- query(expr)
  - expr:查询字符串

```
data.query("open<24 & open>23").head()
```

2. 筛选

- isin([values])

```
# 可以指定值进行一个判断，从而进行筛选操作
data[data["open"].isin([23.53, 23.85])]
```